

Data Analysis with Python 2024–2025

Parte 2: Python Soluções dos Exercícios

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**. As soluções apresentadas a seguir foram elaboradas para fins didáticos, seguindo os enunciados dos exercícios propostos no curso original.

Conteúdo

1 Soluções - Capítulo 2

Exercício 2.1 – Inteiros entre colchetes

Solução proposta

```
import re

def integers_in_brackets(s):
    padrao = re.compile(r'\[\s*([+-]?\d+)\s*\]')
    return [int(x) for x in padrao.findall(s)]

def main():
    texto = " afd [asd] [12 ] [a34] [ -43 ]tt [+12]xxx"
    print(integers_in_brackets(texto))    # [12, -43, 12]

if __name__ == "__main__":
    main()
```

Exercício 2.2 – Listagem de arquivos

Solução proposta

```
import re

def file_listing(caminho="src/listing.txt"):
    padrao = re.compile(
        r'^\S+\s+\d+\s+\S+\s+\S+\s+(\d+)\s+'
        r'(\w+)\s+(\d+)\s+(\d+):(\d+)\s+(.+)$'
    )
    resultado = []
    with open(caminho, "r") as f:
        for linha in f:
            m = padrao.match(linha.rstrip("\n"))
            if m:
                tamanho, mes, dia, hora, minuto, nome = m.groups()
                resultado.append((int(tamanho), mes, int(dia),
                                int(hora), int(minuto), nome))

    return resultado

def main():
    for item in file_listing():
        print(item)

if __name__ == "__main__":
    main()
```

Exercício 2.3 – Vermelho, verde, azul (RGB)

Solução proposta

```
import re

def red_green_blue(caminho="src/rgb.txt"):
    padrao = re.compile(r'^\s*(\d+)\s+(\d+)\s+(\d+)\s+(.+?)\s*$')
    resultado = []
    with open(caminho, "r") as f:
        linhas = f.readlines()[1:] # remove a primeira linha
        for linha in linhas:
            m = padrao.match(linha)
            if m:
                r, g, b, nome = m.groups()
                resultado.append(f"{r}\t{g}\t{b}\t{nome}")
    return resultado

def main():
    lista = red_green_blue()
    if lista:
        print(lista[0]) # 255\t250\t250\tsnow

if __name__ == "__main__":
    main()
```

Exercício 2.4 – Frequência de palavras

Solução proposta

```
def word_frequencies(nome_do_arquivo):
    pontuacao = "!"#$%&'()*,-./:;?@[ ]_""
    frequencias = {}
    with open(nome_do_arquivo, "r", encoding="utf-8") as f:
        for linha in f:
            for palavra in linha.split():
                palavra = palavra.strip(pontuacao)
                if palavra:
                    frequencias[palavra] = frequencias.get(palavra, 0) + 1
    return frequencias

def main():
    frequencias = word_frequencies("alice.txt")
    for palavra, contagem in frequencias.items():
        print(f"{palavra}\t{contagem}")

if __name__ == "__main__":
    main()
```

Exercício 2.5 – Resumo estatístico

Solução proposta

```
import sys
import math

def summary(nome_do_arquivo):
    numeros = []
    with open(nome_do_arquivo, "r") as f:
        for linha in f:
            try:
                numeros.append(float(linha))
            except ValueError:
                continue # ignora linhas que nao representam
                        numeros

    n = len(numeros)
    soma = sum(numeros)
    media = soma / n if n > 0 else 0.0

    if n > 1:
        variancia = sum((x - media) ** 2 for x in numeros) / (n -
            1)
        desvio_padrao = math.sqrt(variancia)
    else:
        desvio_padrao = 0.0

    return (soma, media, desvio_padrao)

def main():
    for nome_do_arquivo in sys.argv[1:]:
        soma, media, desvio = summary(nome_do_arquivo)
        print(f"File: {nome_do_arquivo} Sum: {soma:.6f} "
            f"Average: {media:.6f} Stddev: {desvio:.6f}")

if __name__ == "__main__":
    main()
```

Exercício 2.6 – Contagem de arquivo

Solução proposta

```
import sys

def file_count(nome_do_arquivo):
    linhas = 0
    palavras = 0
    caracteres = 0
    with open(nome_do_arquivo, "r") as f:
        for linha in f:
            linhas += 1
            palavras += len(linha.split())
            caracteres += len(linha)
    return (linhas, palavras, caracteres)

def main():
    for nome_do_arquivo in sys.argv[1:]:
        linhas, palavras, caracteres = file_count(nome_do_arquivo)
        print(f"{linhas}\t{palavras}\t{caracteres}\t{
            nome_do_arquivo}")

if __name__ == "__main__":
    main()
```

Exercício 2.7 – Extensões de arquivo

Solução proposta

```
def file_extensions(nome_do_arquivo):
    sem_extensao = []
    por_extensao = {}

    with open(nome_do_arquivo, "r") as f:
        for linha in f:
            nome = linha.strip()
            if not nome:
                continue
            if "." in nome:
                extensao = nome.rsplit(".", 1)[1]
                por_extensao.setdefault(extensao, []).append(nome)
            else:
                sem_extensao.append(nome)

    return (sem_extensao, por_extensao)

def main():
    sem_extensao, por_extensao = file_extensions("src/filenames.txt")
    print(f"{len(sem_extensao)} files with no extension")
    for extensao in sorted(por_extensao):
        print(f"{extensao}\t{len(por_extensao[extensao])}")

if __name__ == "__main__":
    main()
```

Exercício 2.8 – Prepend

Solução proposta

```
class Prepend:
    def __init__(self, start):
        self.start = start

    def write(self, s):
        print(self.start + s)

def main():
    p = Prepend("+++ ")
    p.write("Hello")    # +++ Hello

if __name__ == "__main__":
    main()
```

Exercício 2.9 – Números racionais

Solução proposta

```
from math import gcd

class Rational:
    def __init__(self, numerador, denominador=1):
        if denominador == 0:
            raise ZeroDivisionError("0 denominador nao pode ser zero")
        if denominador < 0:
            numerador = -numerador
            denominador = -denominador
        d = gcd(abs(numerador), denominador)
        d = d if d != 0 else 1
        self.num = numerador // d
        self.den = denominador // d

    def __add__(self, outro):
        return Rational(self.num * outro.den + outro.num * self.den,
                        self.den * outro.den)

    def __sub__(self, outro):
        return Rational(self.num * outro.den - outro.num * self.den,
                        self.den * outro.den)

    def __mul__(self, outro):
        return Rational(self.num * outro.num, self.den * outro.den)

    def __truediv__(self, outro):
        return Rational(self.num * outro.den, self.den * outro.num)

    def __eq__(self, outro):
        return self.num == outro.num and self.den == outro.den

    def __lt__(self, outro):
        return self.num * outro.den < outro.num * self.den

    def __gt__(self, outro):
        return self.num * outro.den > outro.num * self.den

    def __le__(self, outro):
        return self < outro or self == outro

    def __ge__(self, outro):
        return self > outro or self == outro

    def __ne__(self, outro):
        return not self == outro

    def __str__(self):
        return f"{self.num}/{self.den}"

    def __repr__(self):
        return f"Rational({self.num}, {self.den})"

def main():
    r1 = Rational(1, 4)
    r2 = Rational(2, 3)
    print(r1 + r2)      # 11/12
    print(r1 - r2)      # -5/12
```

Exercício 2.10 – Extrair números

Solução proposta

```
def extract_numbers(s):
    numeros = []
    for palavra in s.split():
        try:
            numeros.append(int(palavra))
        except ValueError:
            try:
                numeros.append(float(palavra))
            except ValueError:
                continue
    return numeros

def main():
    print(extract_numbers("abd 123 1.2 test 13.2 -1"))
    # [123, 1.2, 13.2, -1]

if __name__ == "__main__":
    main()
```